

Question 1 Answer:

You can't override a sealed property or method because sealed explicitly prevents further overriding, locking the implementation at that level.

Question 2 Answer:

Static members belong to the class itself and are shared, while object members belong to individual instances.

Question 3 Answer:

No, not all operators can be overloaded (e.g., =, &&, ||, ?:, new) because some operators are fundamental to C#'s behavior.

Question 4 Answer:

You should change the underlying type of an enum when memory optimization or interop with external systems (like databases or APIs) requires a smaller or specific type.

Question 5 Answer:

A static class can't have instance constructors because you can't instantiate static classes — they're meant to hold only static members.

Question 6 Answer:

Enum.TryParse avoids exceptions on invalid input and provides safe parsing with a success/failure result, unlike direct parsing methods.

Question 7 Answer:

Overriding Equals compares values/content, while == by default checks references for classes and bitwise equality for structs unless overloaded.

Question 8 Answer:

Overriding ToString makes debugging and logging easier by giving meaningful string representations of objects.

Question 9 Answer:

Yes, generics can be constrained.

Question 10 Answer:

Generic methods apply type parameters only to a single method, while generic classes apply type parameters to the entire class.

Question 11 Answer:

A generic swap method is preferable because it works for all types, avoiding duplicate custom implementations.

Question 12 Answer:

Overriding Equals in Department ensures comparisons check logical equality (like Name), improving accuracy in searches.

Question 13 Answer:

== is not implemented by default for structs to avoid confusion, since structs are value types and need explicit equality logic.